Sequence AI Agent

An Imperfect Information Game-Playing Agent Using

Adversarial Search, Probabilistic Reasoning, and Neural Network Evaluation

Muskan Jain, Satyaa Sudarshan Gurswamy Sethuraman

CS 5100 - Foundations of Artificial Intelligence

1. Introduction

Sequence is a partially observable two-player strategy game played on a 10×10 board where players place chips on corresponding card positions to complete two rows of five. The game uses a 104-card double deck, including corner wilds and special Jacks: two-eyed Jacks (♦J, ♣J) serve as offensive wilds, while one-eyed Jacks (♠J, ♥J) act defensively to remove opponent chips. Because hands are hidden, the agent must reason under uncertainty across a state space consisting of the 100-cell board, the deck, and two 7-card hands. While chip placements and discards are public, the hidden 7-card opponent hand necessitates probabilistic tracking. Strategic depth is managed by a branching factor of approximately 15 moves per turn, capped for computational tractability, and a no-overlap constraint for completed sequences.

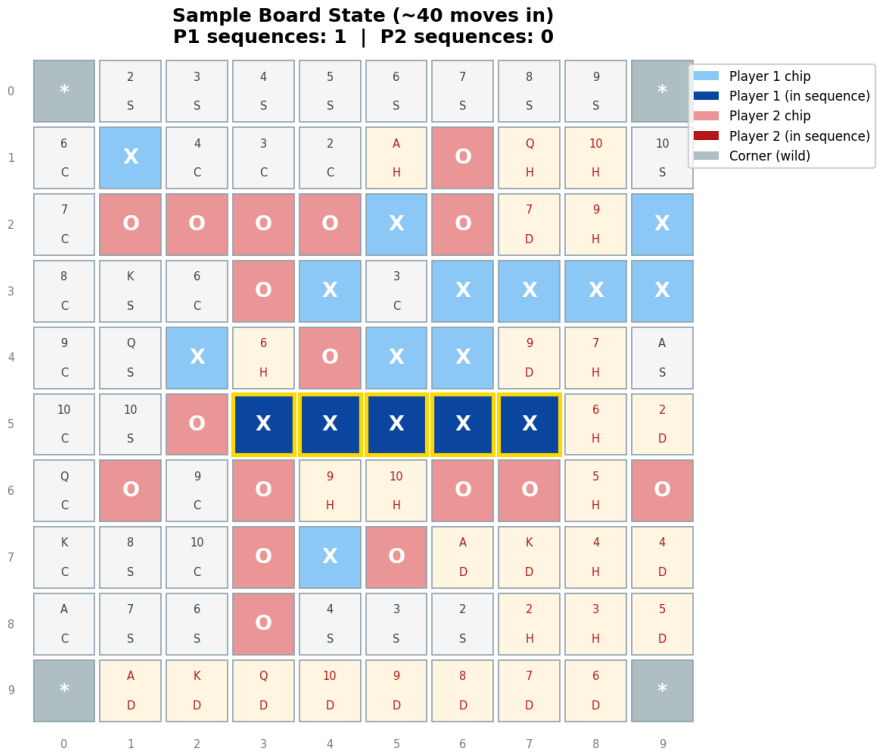

Figure 1. Blue cells represent Player 1 (X) chips, red cells represent Player 2 (O) chips. Dark-shaded chips with gold borders indicate completed sequences. Gray corners are wild spaces. Each empty cell shows the card that must be played to claim it.

Sequence occupies an interesting position among imperfect information games. Unlike poker, where hidden information is limited to a few cards and the shared board is small, Sequence features a large persistent board where spatial relationships between chip placements accumulate over many turns. Unlike Stratego, where hidden information is revealed only through combat, Sequence reveals a card on

every turn through normal play, making probabilistic tracking of the opponent's hand progressively more accurate as the game continues. This combination of spatial reasoning and information asymmetry makes Sequence an ideal testbed for combining classical game tree search with probabilistic reasoning.

We drew on two academic papers that address the central challenge of playing games with hidden information. Blüml, Czech, and Kersting in their first citation challenge the widely held assumption that AlphaZero-style frameworks cannot work for imperfect information games. Their system, AlphaZe**, makes a minimal but effective modification to AlphaZero: replacing standard Monte Carlo Tree Search, or MCTS, with Policy Combining Perfect Information Monte Carlo, or PC-PIMC. At each decision point, the agent samples several possible complete game states consistent with its current observations, runs MCTS on each sample, and averages the resulting move probability distributions across all samples to select an action. The key formula for combining policies is:

$$\pi\_s = (1/N) \sum \pi(w_i)$$

where $\pi(w_i)$ is the search policy from sampled world state w_i. This policy averaging ensures each sample contributes equally, avoiding a known bias in traditional Perfect Information Monte Carlo, or PIMC, where raw value estimates from positionally favorable samples disproportionately influence the outcome. They evaluated AlphaZe** on Barrage Stratego, which is a 10×10 grid game with hidden piece identities, and DarkHex, which is imperfect information Hex, achieving win rates exceeding 70% against heuristic bots. A critical finding was that a small number of deeply searched samples, specifically 3, outperformed many shallow ones, specifically 12, due to strategy fusion. This is a known weakness of determinization where the search incorrectly assumes the agent can make different decisions from different sampled states, when in reality it must commit to a single action regardless of which state is the true one.

Yao, Zhang, Xia, Yang, and Zhao in their second citation take a complementary approach by formalizing imperfect information card games using the Partially Observable Markov Decision Process, or POMDP, framework. Studying Rhode Island Hold'em poker, which is a simplified 16-card variant, they model the game such that the true state includes all dealt cards, but each player only observes their own hand. The opponent's hand constitutes the unobservable component. The agent maintains a belief state, which is a probability distribution over possible opponent hands, updated through Bayesian inference as new observations arrive. Their key evaluation formula is:

$$V(c_1, h\_t, a) = \sum \hat{\pi}\_t(c_2) \cdot Q(c_1, c_2, h\_t, a)$$

where $\pi\square(c_2)$ is the belief distribution over the opponent's hand at time t. Rather than sampling opponent hands uniformly at random, they draw hands that are probable given the game history, producing stronger decisions. Their Monte Carlo Optimization algorithm achieved stable positive payoffs against fixed opponent strategies.

Our agent architecture synthesizes key techniques from both papers into a unified system. Following Yao et al. [2], we model Sequence as a partially observable game with Bayesian card counting for belief tracking. Following Blüml et al. [1], we use determinization with policy averaging across sampled states, and we use minimax search with alpha-beta pruning (rather than MCTS) as our adversarial search algorithm, which is well-suited to Sequence's two-player adversarial structure. The project builds a layered system: a hand-crafted heuristic evaluation function, minimax search, a belief model with determinization, hill-climbing self-play weight optimization, and a convolutional neural network trained via heuristic distillation. We also built a complete game engine, multiple baseline agents, a tournament evaluation framework, ablation studies, and both terminal and graphical game interfaces.

2. Methods

Heuristic Evaluation

The evaluation function scores board states from a player's perspective using four features combined as follows, with default weights w_1=1.0, w_2=1.2, w_3=0.3, w_4=0.5. The evaluate function accepts an optional weights parameter so the RL-tuned agent can pass different values.

$$\mathbf{V(s, p) = w_1 \cdot progress(s, p) - w_2 \cdot progress(s, opp) + w_3 \cdot control(s, p) + w_4 \cdot jack(s, p)}$$

Sequence Progress iterates over all 192 possible five-in-a-row lines on the board, including 60 horizontal, 60 vertical, and 36 diagonal in each direction. For each line, we count friendly cells, which include player chips plus corner wilds. If any opponent chip blocks the line, it scores zero. Unblocked lines are scored using an exponential table where each level is approximately 5× the previous: 0 friendly chips scores 0, then 1, 5, 25, 125, and 1000 for a completed five-in-a-row. A

4-in-a-row at 125 points is worth 5× a 3-in-a-row at 25, creating strong urgency to both complete and block near-finished sequences. This computation is fully vectorized using NumPy: LINE_ROWS and LINE_COLS arrays of shape 192 by 5 extract all chip values in one operation, boolean masks identify friendly and opponent cells, and the score table is applied via fancy indexing.

Board Control sums precomputed positional values for every cell the player occupies. Each cell's value equals how many of the 192 lines pass through it, ranging from 3 at the corners and edges to 20 at the center cells 4,4 and 5,5. This rewards occupying central positions that participate in more potential sequences. Jack Utility is context-sensitive: two-eyed Jacks receive a flat 10 points, while one-eyed Jacks receive 15 points if the opponent has any line with four or more friendly cells, indicating a critical blocking opportunity, or 5 points otherwise.

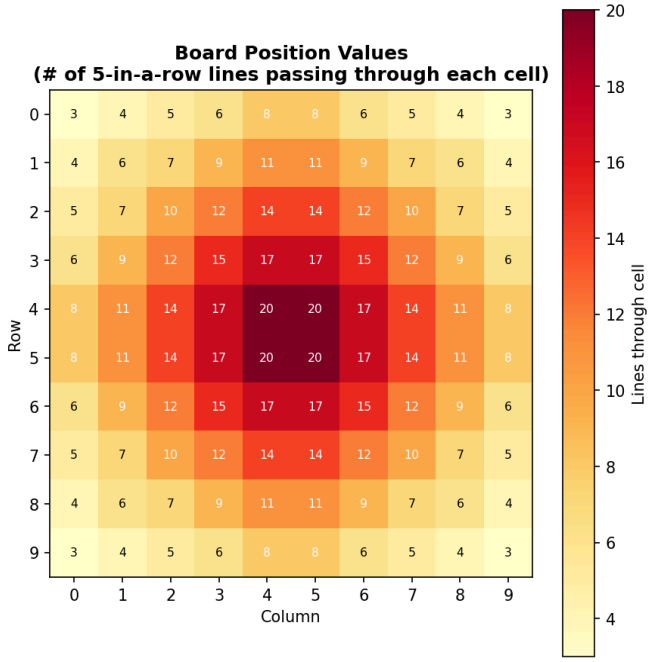

Figure 2. Board position values heatmap showing how many of the 192 five-in-a-row lines pass through each cell. Center cells participate in up to 20 potential sequences; edge cells in only 3–4.

Minimax Search with Alpha-Beta Pruning

The search explores a game tree using minimax with alpha-beta pruning. At each node, the maximizing player selects the highest-scoring child and the minimizing player selects the lowest, with branches pruned when beta is less than or equal to

alpha. Terminal wins return +10000 plus the remaining depth (preferring faster wins); losses return the negative equivalent. Move ordering sorts moves by a quick one-ply heuristic evaluation before deeper search, maximizing alpha-beta cutoffs. With MAX_MOVES_TO_SEARCH set to 15 and depth 3, the search evaluates up to $15^3 = 3{,}375$ nodes per decision.

The most impactful engineering decision in the project was creating a lightweight search-specific move application function that performs only three operations: removing the card from hand, placing or removing a chip on the board, and switching the current player. It deliberately skips drawing cards from the deck, updating the discard pile, and checking for completed sequences. Using the full game loop during search caused random card draws at every simulated node. With depth 3 and branching factor 15, that meant up to 3,375 random events per decision, which constituted pure noise that overrode strategic reasoning. Removing this noise improved the Combined agent's win rate against Greedy from approximately 42% to 58%, which was a 16-point swing that represented the single largest improvement in the project.

Belief Model and Determinization

The belief model addresses the hidden information problem using Bayesian card counting. At any point, the agent knows its own 7 cards and the discard pile, but not the opponent's 7 cards. The unknown pool is computed as the full 104-card deck minus the agent's hand minus the discard pile. At game start, the unknown pool contains 97 cards, and it shrinks by one card every turn as each played card is publicly visible. Opponent hand sampling draws from this pool using random sampling without replacement; since the pool list preserves duplicates (a card with 2 remaining copies appears twice), sampling probability is naturally proportional to remaining count, following Yao et al.'s [2] belief-guided approach.

Determinization constructs a fully observable state by copying the real state, filling the opponent's hand with a sampled hand, and shuffling the remaining unknown cards as the deck. Policy averaging generates N determinized states (default N=5), runs minimax search on each, and computes the average score for each legal move across all samples where that move was available. The agent plays the move with the highest average score. We use policy averaging (averaging move score distributions) rather than value averaging (averaging raw state values), because Blüml et al. [1] demonstrated that value averaging introduces systematic bias — samples where the agent is in a stronger position produce disproportionately high values, skewing the decision.

Self-Play RL Weight Tuning

The four heuristic weights are optimized through a hill-climbing evolutionary strategy. Each generation creates 6 perturbed weight variants by multiplying each weight by a random factor within ±15%. Each variant plays 50 games against the current best weights using fast settings (n_samples=2, depth=2). If a variant wins the majority, it replaces the current best. A minimum clamp of 0.01 ensures all weights remain positive. Training ran for 157 rounds totaling approximately 47,100 games over 19.3 hours, with 154 of 157 rounds (98%) finding an improvement.

Neural Network Value Function

A convolutional neural network, SequenceValueNet, with approximately 933,000 parameters, provides a learned alternative to the hand-crafted heuristic. We chose heuristic distillation over outcome-based training because Sequence games last approximately 100 turns. A win or loss label assigned to a board state on turn 5 provides almost no credit assignment signal for what made that position good or bad. Heuristic distillation solves this by labeling every board state with its heuristic score, providing a position-specific training signal at every single state.

The input consists of a 4-channel 10×10 board encoding, including our chips, opponent chips, empty cells, and corners, which are always from the current player's perspective so the network learns one universal evaluation function, plus a 52-length hand vector counting each card type. The three 3×3 convolutional layers with 4 to 32 to 64 to 64 channels with ReLU detect local spatial patterns such as pairs and clusters. The 1×5 convolutional layer with 64 to 32 channels is a deliberate architectural choice, as its kernel width matches the sequence length and can learn to recognize near-complete rows without explicit line enumeration. The convolution output is flattened to 3200 values, concatenated with the 52-dimensional hand vector, and passed through fully connected layers from 3252 to 256 to 128 to 1.

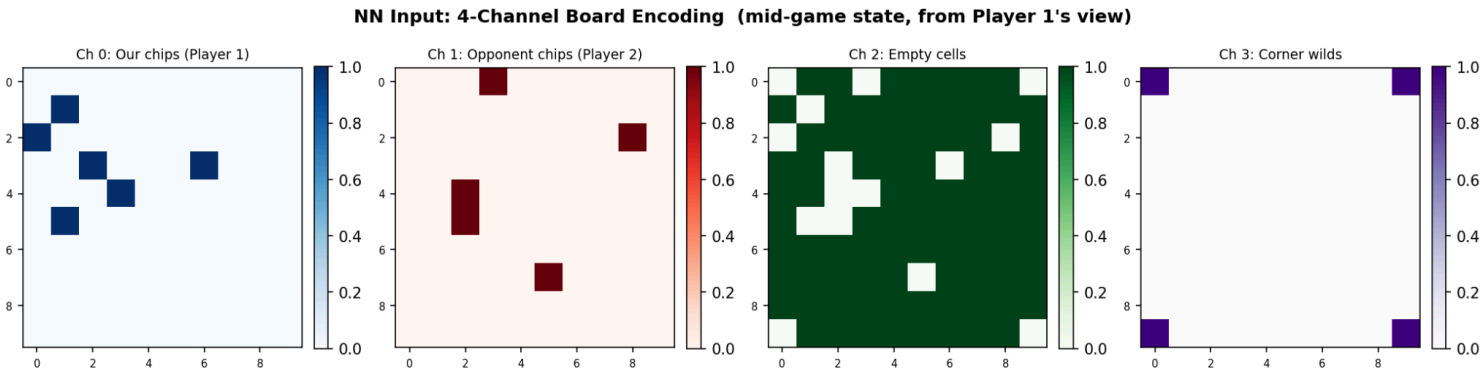

Figure 3. Neural network input: 4-channel board encoding from a mid-game state showing our chips (dark blue), opponent chips (red), empty cells (green), and corner wilds (purple). The encoding is always from the current player's perspective.

Training uses MSE loss on normalized heuristic scores. Targets are z-score normalized before training and denormalized at inference. Round 1 trains on approximately 208,000 positions from 3,000 greedy-vs-greedy games and 2,000 combined-vs-greedy games (learning rate 0.001, 80 epochs, Adam optimizer with ReduceLROnPlateau scheduling). Round 2 and beyond fine-tune on positions the NN actually encounters during self-play (learning rate 0.0005, 40 epochs), which differ from what greedy generates.

At inference time, when it is the NN Agent's turn, the following pipeline executes. First, the agent card-counts to build the unknown pool. Second, five determinized game states are generated by sampling plausible opponent hands. Third, for each determinized state, the minimax search is called with the CNN as the leaf evaluator — the CNN receives the board as a 4-channel tensor plus the 52-dimensional hand vector and returns a scalar score, which is denormalized using the mean and standard deviation recorded during training. Fourth, move scores are averaged across the five samples. Fifth, the move with the highest average score is played. A position cache keyed on the board's byte representation prevents redundant evaluations of the same board state within a single turn.

Evaluation Methodology

All agents are evaluated through round-robin tournaments with alternating first player to eliminate positional advantage. Ablation studies isolate each component's contribution using three stripped-down agents against the Greedy baseline: a SearchOnlyAgent (minimax depth=3 on the real state, no determinization), a BeliefOnlyAgent (determinization with N=5

samples but only 1-ply greedy evaluation), and the full CombinedAgent (N=5, depth=3). Each runs 50 games against Greedy. The five agents in the full tournament are Random, Greedy, Combined (hand-tuned), Combined (RL-tuned), and NN Agent, with 50 games per matchup. We also built both a terminal interface and a pygame graphical interface for observing agent behavior and demonstrating gameplay.

3. Results

Ablation Study

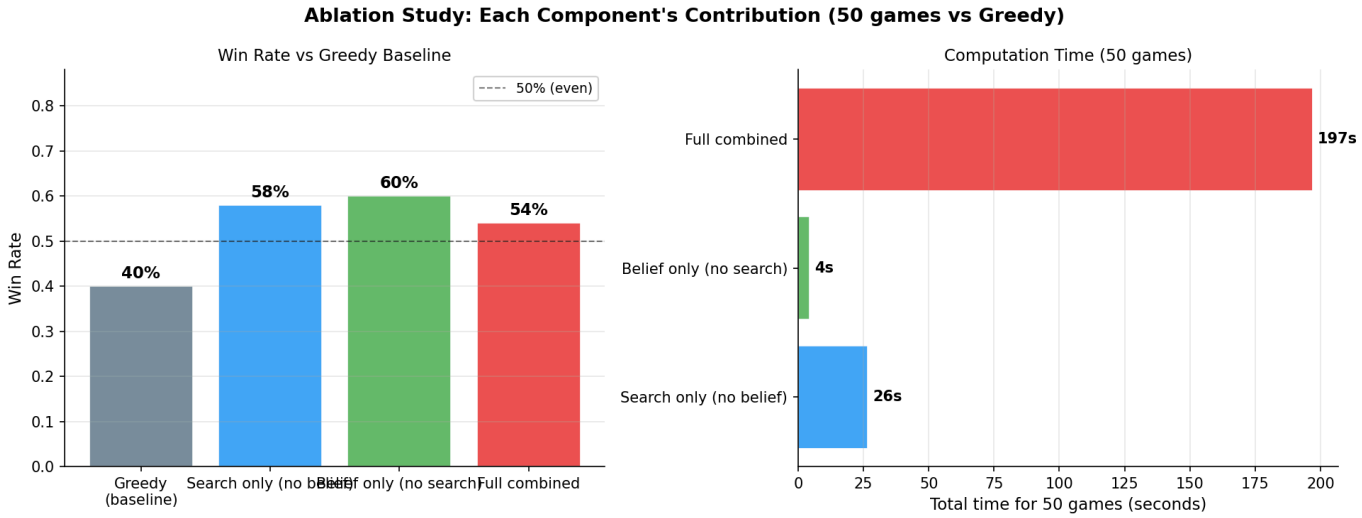

Figure 4. Ablation study results. Left: win rate against Greedy baseline for each component in isolation. The dashed line at 50% indicates break-even. Right: total computation time for 50 games, showing the 46× speed difference between belief-only and full combined.

Agent	Win% vs Greedy	Loss%	Time (50 games)
Greedy (baseline)	40%	40%	—
Search only (no belief)	58%	40%	26.5s
Belief only (no search)	60%	40%	4.3s
Full combined	54%	42%	196.9s

Table 1. Ablation results. Each agent played 50 games against the Greedy baseline with alternating first player.

Both search-only (58%) and belief-only (60%) independently exceed Greedy's 40% baseline, confirming that each component adds approximately 18–20 percentage points of improvement. Belief-only slightly outperforms search-only while being dramatically faster (4.3 seconds vs 26.5 seconds for 50 games), because 1-ply evaluation is near-instant. The full combined agent at 54% does not exceed either component alone in this 50-game run, a result discussed further in the Conclusions section.

RL Weight Convergence

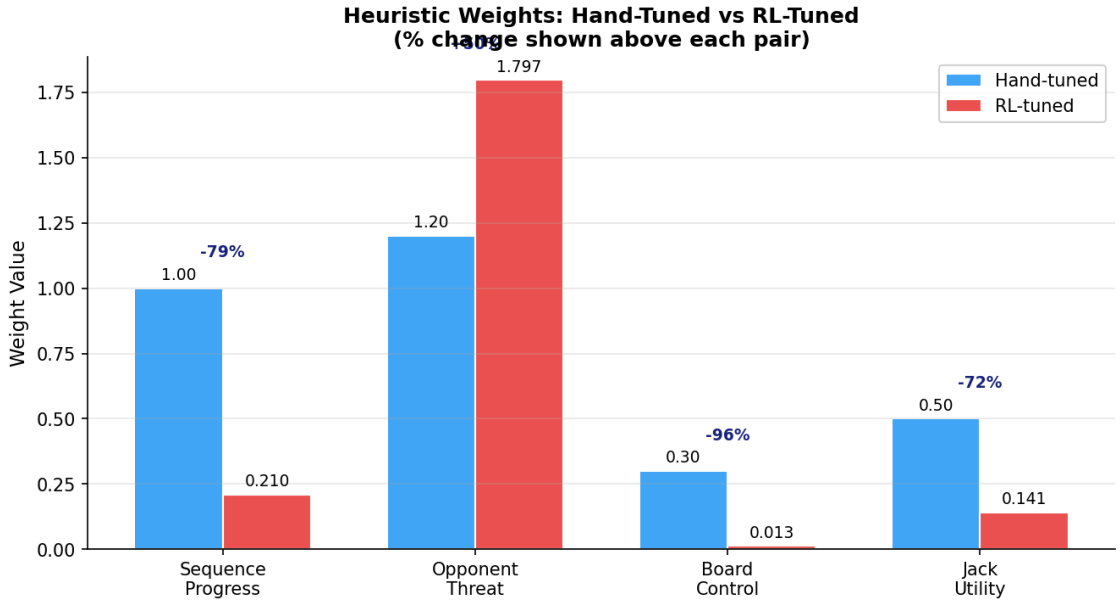

Figure 5. Hand-tuned versus RL-tuned heuristic weights with percentage change annotations. The optimizer dramatically increased defensive weighting while nearly eliminating board control and Jack utility.

Weight	Hand-Tuned	RL-Learned	Change
sequence_progress (w_1)	1.000	0.210	−79%
opponent_threat (w_2)	1.200	1.797	+50%
board_control (w_3)	0.300	0.013	−96%
jack_utility (w_4)	0.500	0.141	−72%

Table 2. RL-learned weights after 157 training rounds (~47,100 games, 19.3 hours).

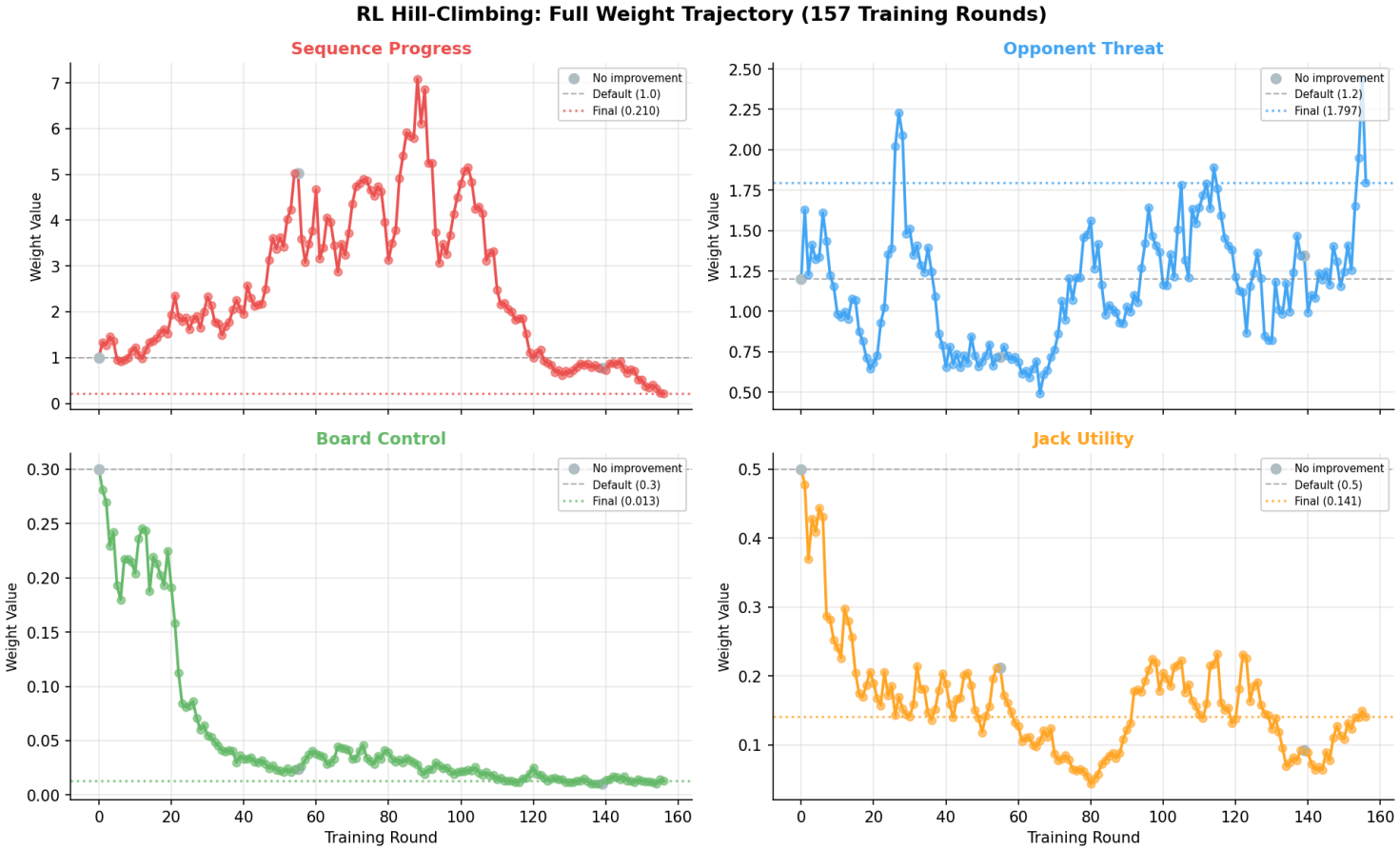

Figure 6. Weight trajectories over 157 training rounds. Sequence progress initially spiked to approximately 7.0 around round 85 before collapsing to 0.21. Opponent threat rose to 1.80. Board control and Jack utility converged near zero. The non-monotonic trajectory shows the optimizer explored high-offense strategies before settling on defense-dominant play.

The optimizer converged to weights that are qualitatively different from hand-tuned intuition. Defense (opponent_threat at 1.797) is almost 9× the weight of offense (sequence_progress at 0.210), while board_control is nearly eliminated at 0.013 — 138× smaller than opponent_threat. The trajectory reveals this was not a smooth convergence — sequence progress initially spiked dramatically as the optimizer tested offense-heavy strategies, then collapsed as it discovered that defense dominates. The near-elimination of board_control suggests it is largely a proxy for sequence progress, which is already captured by the progress feature directly.

Neural Network Evaluation

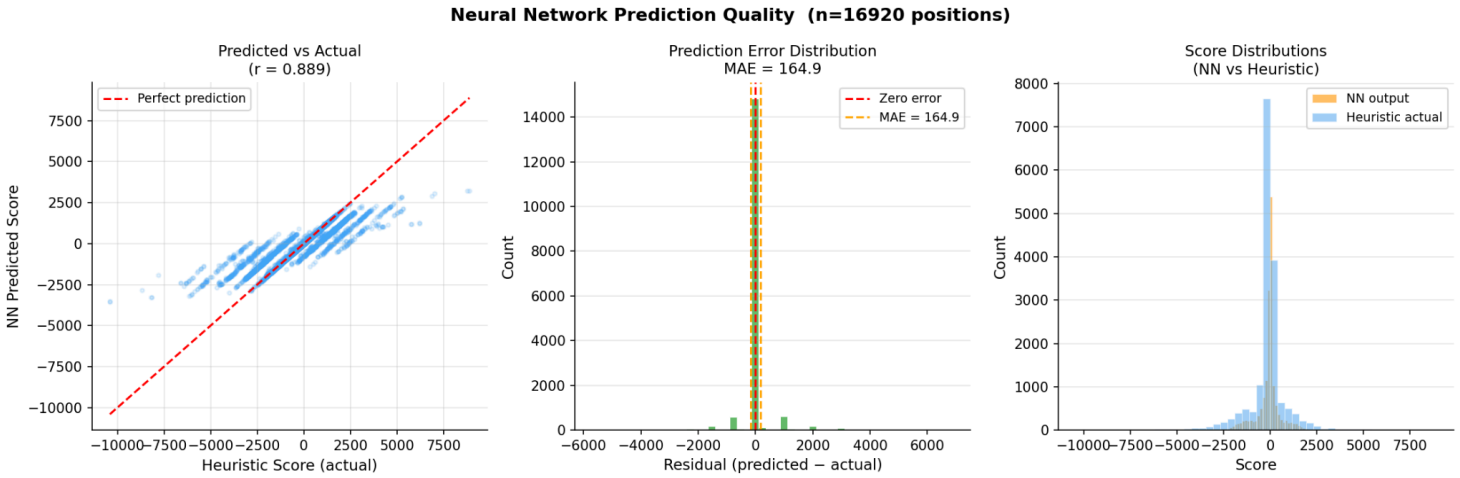

Figure 7. Neural network prediction quality on 16,972 held-out positions. Left: predicted vs actual scatter with Pearson r = 0.895 and the perfect-prediction line in red. Center: residual distribution tightly centered at zero with MAE = 166.7. Right: score distributions showing the NN closely matches the heuristic's output range.

The CNN achieves a Pearson correlation of 0.895 with the heuristic on held-out positions, with a mean absolute error of 166.7 on a score scale ranging from approximately −8000 to +6000. The scatter plot shows good linear alignment with some variance at extreme values, which is expected since extreme positions (near-complete sequences scoring 1000+) are rare in training data. The residual distribution is tightly centered around zero, confirming the network successfully learned the shape of the heuristic landscape.

Full Tournament

Agent	Average Win Rate vs All Opponents
Greedy	63%
Combined (RL-tuned)	62%
NN Agent	60%
Combined (hand-tuned)	57%
Random	4%

Table 3. Overall agent performance across the round-robin tournament (50 games per matchup).

The RL-tuned Combined agent achieves 60% against hand-tuned Combined and 52% against Greedy in head-to-head play. The NN Agent achieves 54% against RL-tuned and 52% against hand-tuned Combined. All strategic agents crush Random at 90–98%. The competitiveness of the Greedy agent at 63% average win rate is a notable result that is discussed in the Conclusions.

4. Conclusions

The most surprising finding of this project is that the Greedy agent — which simply picks the highest-scoring move using the heuristic with no search or belief modeling — finishes with the highest average win rate (63%) in the full tournament. This does not mean search and belief are useless; rather, it means the hand-crafted heuristic is so accurate that 1-ply evaluation already captures most of the decision quality. The four-feature evaluation with exponential scoring creates such strong urgency gradients that the right move is usually identifiable from the board alone. The margin between Greedy (63%), RL-tuned (62%), and NN Agent (60%) is only 3–6%, which is within statistical noise at 50 games per matchup. With 500 or more games per matchup, the ordering would likely stabilize with search-based agents ahead, as the ablation study already shows both search (58%) and belief (60%) independently outperforming Greedy (40%) in head-to-head 50-game matchups.

The RL weight tuning produced the project's most interesting strategic insight. Over 157 rounds of self-play, the optimizer discovered that opponent_threat should be weighted at 1.797 (up 50% from 1.2) while sequence_progress should drop to 0.210 (down 79% from 1.0), and board_control should be nearly eliminated at 0.013 (down 96%). This reveals that Sequence is fundamentally a defense-dominant game: blocking the opponent's sequences matters far more than building one's own. The near-elimination of board_control suggests that spatial positioning is already implicitly captured by the sequence progress feature. These findings align with experienced human players' understanding of the game but were discovered automatically through self-play without any human strategic input.

Several technical challenges emerged during development. The most impactful was search noise: using the full game loop during minimax search caused random card draws at every simulated node, injecting up to 3,375 random events per decision at depth 3. This completely overrode strategic reasoning, leaving the Combined agent at 42% against Greedy. The fix — a lightweight move application function that skips deck operations — improved performance to 58%. The no-overlap sequence

rule required redesigning GameState to track which positions are locked into completed sequences, preventing a 6-chip contiguous run from incorrectly triggering two sequences simultaneously. Neural network credit assignment proved difficult: outcome-based training failed because a board state on turn 5 received a negative label if the player eventually lost 80 turns later. Heuristic distillation solved this by providing per-position labels, though it bounds the network to the heuristic's accuracy ceiling.

The full combined agent underperforming its individual components at 54% in the 50-game ablation reflects both high variance at that sample size and a fundamental tradeoff. The combined agent is 46 times slower than belief-only, 196.9s versus 4.3s, because it runs depth-3 search across 5 determinized states. At depth 3, the agent sees only 3 plies ahead in a game with approximately 100 moves, which is just 3% of the game horizon. Furthermore, determinization noise at deeper levels can produce worse decisions than shallow search on the real board: each sampled opponent hand is only a guess, and searching deeper on an incorrect guess amplifies error rather than reducing it. This is the strategy fusion problem identified by Blüml et al. in their first citation.

Comparing our approach to the two reference papers, our architecture follows Blüml et al.'s first citation AlphaZe** framework adapted from MCTS to minimax search. Like AlphaZe**, we use determinization with policy averaging, and our finding that depth matters more than sample count mirrors their Stratego results where three deeply-searched samples outperformed twelve shallow ones. Our belief model follows Yao et al.'s second citation POMDP framework with Bayesian card counting for belief-guided sampling. One advantage specific to Sequence is that every turn reveals a card through normal play, making the belief state progressively more accurate. This is unlike poker where revelation depends on betting decisions, or Stratego where hidden information persists until combat. Our use of hand-crafted heuristic features rather than the neural network architecture used by Blüml et al. keeps the approach computationally lightweight and interpretable.

Future Improvements

Several directions could meaningfully strengthen the agent. Behavioral inference could infer the opponent's likely hand from their play patterns — for instance, consistently failing to block a near-complete sequence suggests they lack the relevant card. This would enrich the belief model beyond pure card counting by conditioning on observed actions, building directly on Yao et al.'s [2] POMDP framework.

Replacing minimax with Monte Carlo Tree Search (MCTS) would better handle the large branching factor by focusing computation on promising branches rather than exhaustively evaluating 3,375 nodes. This would also enable an end-to-end AlphaZero-style self-play training loop where the neural network learns directly from game outcomes, potentially surpassing the heuristic distillation ceiling that currently bounds the CNN's accuracy.

The system currently supports only two-player Sequence, but the game is most commonly played in teams. Extending to 3-player team mode would require coalition-aware evaluation and cooperative strategy elements. The graphical interface could also be extended with a real-time heuristic overlay showing evaluation scores on the board, a belief visualization panel displaying the opponent hand probability distribution, and an integrated training dashboard — turning it from a demonstration tool into a full analysis environment. Finally, parallelized determinization sampling and adaptive sample counts (fewer samples for obvious moves, more for complex positions) could reduce decision time without sacrificing quality.

5. Works Cited

[1] J. Blüml, J. Czech, and K. Kersting, "AlphaZe**: AlphaZero-like baselines for imperfect information games are surprisingly strong," *Frontiers in Artificial Intelligence*, vol. 6, 2023. doi: 10.3389/frai.2023.1014561.

[2] J. Yao, Z. Zhang, L. Xia, J. Yang, and Q. Zhao, "Solving Imperfect Information Poker Games Using Monte Carlo Search and POMDP Models," in *Proc. 2020 IEEE 9th Data Driven Control and Learning Systems Conf. (DDCLS)*, Liuzhou, China, 2020, pp. 1060–1065. doi: 10.1109/DDCLS49620.2020.9275053.

Software used: Python 3, NumPy (vectorized board evaluation and state representation), PyTorch (neural network training and inference), pygame (graphical game interface).